Kite — Red social distribuida

Documento técnico · Entrega 1: Microservicios + gRPC + Docker

Diego Beñaldo · Maximiliano Sepulveda

Universidad de La Frontera — Sistemas Distribuidos y Escalables — Octubre 2026

- 1. Descripción del sistema
- 2. Diagrama de arquitectura
- 3. Servicios y contratos
- 4. Modelo de datos
- 5. Casos de uso
- 6. Decisiones técnicas y trade-offs
- 7. Resiliencia y manejo de errores
- 8. Cumplimiento de los requisitos de la entrega
- 9. Cómo levantar y probar el sistema
- 10. Estado y trabajo pendiente

1. Descripción del sistema

Kite es una red social mínima construida como un sistema de microservicios. El sistema permite a una persona **crear su cuenta y su perfil**, **seguir a otras personas**, **consultar el grafo de seguidores** y **recibir y gestionar notificaciones** cuando alguien comienza a seguirla.

Usuarios del sistema. Kite está pensado para dos clases de consumidores:

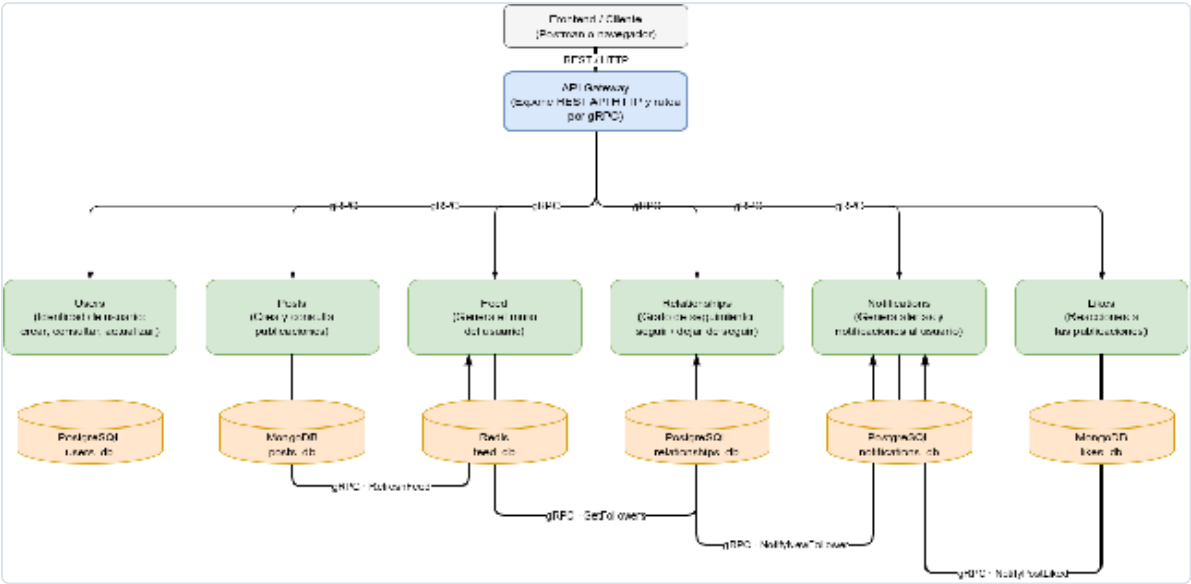
- **Personas usuarias** (a través de un cliente: aplicación, navegador o herramientas como `curl` o Postman), que ejercen las operaciones de negocio: crear cuenta, editar perfil, seguir y dejar de seguir, consultar seguidores y revisar su bandeja de alertas.
- **Clientes integradores** (un futuro frontend u otra aplicación), que consumen la API REST del sistema programáticamente.

Propósito y alcance. El objetivo de esta primera entrega es demostrar una arquitectura de microservicios coherente y funcional de extremo a extremo sobre un dominio con suficiente riqueza de relaciones entre entidades. El alcance incluye la **identidad**, el **grafo de seguimiento** y las **notificaciones**; las publicaciones, el muro y las reacciones se incorporarán más adelante sobre la misma arquitectura (ver sección 10). La autenticación queda **fuera del alcance**: la identidad del solicitante se declara en un encabezado, según se justifica en la sección 6.

Característica arquitectónica central. El sistema tiene **una sola superficie externa**: un **API Gateway** que habla HTTP/REST hacia afuera y gRPC hacia adentro. Ningún servicio de negocio es alcanzable desde fuera de la red de Docker. Cada servicio es **dueño exclusivo de su propia base de datos** y nunca lee la de otro: cuando necesita datos ajenos, se los pide por gRPC al servicio que los posee, o el gateway los **agrega** en una sola respuesta.

---

2. Diagrama de arquitectura



Arquitectura del sistema Kite: cliente, API Gateway, los servicios de negocio y sus bases de datos

Convenciones del diagrama. Las flechas entre servicios indican la dirección de la llamada y el protocolo utilizado (todas entre servicios son **gRPC**; la única entrada externa es **REST/HTTP**). Las flechas hacia una base de datos son accesos **locales** del servicio dueño a su propia base, no comunicaciones entre servicios: no existe ninguna flecha de servicio a servicio que atraviese una base de datos.

Servicio	Lenguaje	Base de datos	Responsabilidad	Puerto	¿Expuesto al exterior?
API Gateway	Go	—	Única superficie REST; traduce a gRPC y **agrega** lecturas	8080	**Sí** (único puerto publicado)
Users	Go	PostgreSQL `users_db`	Identidad: crear, consultar y editar usuarios	50051	No
Relationships	Go	PostgreSQL `relationships_db`	Grafo de seguimiento	50052	No
Notifications	Python	PostgreSQL `notifications_db`	Alertas y bandeja del usuario	50053	No

Los servicios **Posts** (MongoDB), **Feed** (Redis) y **Likes** (MongoDB) forman parte del diseño objetivo y ya están dibujados en el diagrama; se implementarán en la siguiente iteración sobre la misma arquitectura.

Aislamiento de red. En `docker-compose.yml` los servicios y las bases viven en una red `backend` **interna** (sin puertos publicados y sin salida al exterior), y únicamente el API Gateway se conecta además a la red `frontend`, la única publicada al host. El resultado, verificable, es que solo el puerto 8080 responde desde fuera de Docker: los puertos 50051–50053 están cerrados al host.

---

3. Servicios y contratos

El contrato interno es **Protobuf sobre gRPC**; los `.proto` y los stubs generados se versionan dentro de cada servicio (y el gateway mantiene una copia de cada contrato de cliente), de modo que cada imagen se construye desde su propio contexto sin dependencias externas.

3.1 Servicio Users (Go — `users_db`)

Administra la entidad **Usuario**. El `user_id` es un UUID generado por la base de datos del servicio, nunca por el cliente.

Método gRPC	Entrada	Salida	Notas
`CreateUser`	`username`, `email`, `display_name`	`User` completo	Valida formato y unicidad
`GetUser`	`user_id`	`User` completo	`NotFound` si no existe
`UpdateUser`	`user_id`, `display_name?`, `email?`	`User` completo	Parche: solo modifica lo enviado

La entidad `User` expone `user_id`, `username`, `email`, `display_name`, `created_at` y `updated_at`. El `username` es el **identificador estable** y no se edita.

3.2 Servicio Relationships (Go — `relationships_db`)

Administra el **grafo de seguimiento**: quién sigue a quién. Es la única fuente de verdad de esa relación.

Método gRPC	Entrada	Salida	Notas
`FollowUser`	`follower_id`, `followee_id`	`Follow`	Idempotente; rechaza el auto-seguimiento
`UnfollowUser`	`follower_id`, `followee_id`	(vacío)	`NotFound` si la arista no existía
`GetFollowers`	`user_id`, `limit`, `offset`	`user_ids`, `total`	Paginado
`GetFollowing`	`user_id`, `limit`, `offset`	`user_ids`, `total`	Paginado

`FollowUser` invoca `Notifications.NotifyNewFollower` **solo cuando la arista es nueva** (una repetición idempotente no vuelve a notificar), y lo hace con **degradación elegante**: si Notifications está caído, el seguimiento se persiste igual y solo se registra una advertencia.

3.3 Servicio Notifications (Python — `notifications_db`)

Recibe los **dos disparadores** del sistema y expone la bandeja del usuario.

Método gRPC	Entrada	Salida	Notas
`NotifyNewFollower`	`follower_id`, `followee_id`	`Notification`	Disparado por Relationships

Método gRPC	Entrada	Salida	Notas
`NotifyPostLiked`	`post_id`, `liker_id`, `post_author_id`	`Notification`	Disparado por Likes (siguiente iteración)
`GetUnreadNotifications`	`user_id`, `limit`, `offset`	`notifications`, `total`	Paginado
`MarkAsRead`	`user_id`, `notification_id`	`Notification`	`NotFound` si no es del usuario

El tipo de notificación es un **enum** cerrado del contrato (`NEW_FOLLOWER`, `LIKE`), y `post_id` solo tiene valor en las de tipo `LIKE`.

3.4 API Gateway (Go — REST → gRPC)

Es la **única** superficie externa. Traduce cada petición a una o más llamadas gRPC. **No contiene reglas de negocio**: esas viven en los microservicios; el gateway solo transporta, valida la forma de la petición y **agrega**.

Método REST	Delega en	Notas
`GET /`	(local)	Índice de la API: mapa de rutas y del encabezado `X-User-Id`
`POST /users`	`Users.CreateUser`	`201 Created`
`GET /users/{user_id}`	`Users.GetUser`	`200`
`PATCH /users/{user_id}`	`Users.UpdateUser`	Parche parcial
`POST /users/{user_id}/follow`	`Relationships.FollowUser`	Identidad en `X-User-Id`
`DELETE /users/{user_id}/follow`	`Relationships.UnfollowUser`	`204 No Content`
`GET /users/{user_id}/followers`	`Relationships.GetFollowers`	`limit`, `offset`
`GET /users/{user_id}/following`	`Relationships.GetFollowing`	`limit`, `offset`
`GET /notifications`	**Agrega** `Notifications` + `Users`	Ver sección 5, UC-4
`POST /notifications/{notification_id}/read`	`Notifications.MarkAsRead`	
`GET /health`	(local)	Estado del gateway y de sus backends

Las operaciones que actúan **en nombre de una persona** reciben su identidad en el encabezado **`X-User-Id`**. La raíz (`GET /`) devuelve un índice de la API, y el gateway tolera una barra final de más (`/users/{id}/`) reescribiendo el camino internamente, para que la URL que se teclea a mano nunca dependa de un detalle del enrutador.

---

4. Modelo de datos

Cada servicio posee su propia base y aplica sus migraciones al arrancar (embebidas en el binario o en el paquete, versionadas de forma incremental y solo hacia adelante).

`users_db` · tabla `users`

Columna	Tipo	Restricción
`user_id`	UUID	PK, `DEFAULT gen_random_uuid()`
`username`	TEXT	`UNIQUE`, `NOT NULL`
`email`	TEXT	`UNIQUE`, `NOT NULL`, indexado
`display_name`	TEXT	`NOT NULL`
`created_at`	TIMESTAMPTZ	`NOT NULL`, `DEFAULT now()`
`updated_at`	TIMESTAMPTZ	`NOT NULL`, `DEFAULT now()`

`relationships_db` · tabla `follows`

Columna	Tipo	Restricción
`follower_id`	UUID	PK compuesta, `NOT NULL`
`followee_id`	UUID	PK compuesta, `NOT NULL`, indexado
`created_at`	TIMESTAMPTZ	`NOT NULL`, `DEFAULT now()`

Restricción `CHECK (follower_id <> followee_id)` e índice sobre `followee_id`.

`notifications_db` · tabla `notifications`

Columna	Tipo	Restricción
`id`	UUID	PK, `DEFAULT gen_random_uuid()`
`user_id`	UUID	`NOT NULL` (destinatario)
`type`	VARCHAR(32)	`NOT NULL`, `CHECK IN ('LIKE','NEW_FOLLOWER')`
`source_user_id`	UUID	`NOT NULL` (quien originó la acción)
`post_id`	TEXT	`CHECK`: obligatorio en `LIKE`, nulo en `NEW_FOLLOWER`
`is_read`	BOOLEAN	`NOT NULL`, `DEFAULT false`
`created_at`	TIMESTAMPTZ	`NOT NULL`, `DEFAULT now()`

Índice `(user_id, is_read, created_at DESC)` para la bandeja sin leer.

Entidades de dominio persistidas. El sistema cubre cuatro entidades con persistencia real — `User`, `Follow`, `Notification` y sus claves de relación—, muy por encima del mínimo de dos exigido por la rúbrica. Nada relevante vive en memoria: las bases se guardan en **volúmenes** de Docker, y se verificó que los datos sobreviven a un `docker compose down` seguido de `up`.

Referencias lógicas, no claves foráneas cruzadas. `follows.follower_id`, `follows.followee_id`, `notifications.user_id`, `notifications.source_user_id` y `notifications.post_id` **apuntan** a datos de otros servicios, pero sus bases **no** declaran claves foráneas hacia ellos: una base que pertenece a otro servicio no podría verificar esa existencia, y declararla acoplaría los esquemas. Las reglas de dominio que sí son locales (unicidad de `username` y `email`, no seguirse a sí mismo, forma de la notificación) se refuerzan con restricciones reales **en la base**, como defensa en profundidad además de la validación del servicio.

---

5. Casos de uso

Cada caso se describe en **dos capas**: (1) la vista de la persona usuaria y el resultado esperado, y (2) el detalle técnico, con el recorrido, el orden de comunicación entre servicios y dónde queda persistido.

UC-1 · Crear cuenta y editar el perfil

Capa 1 — vista de usuario. Una persona nueva se registra indicando su `username`, su `email` y su nombre visible. El sistema le asigna un identificador y la deja lista para operar. Más tarde ajusta su nombre visible sin cambiar su `username` (que es su identificador estable). *Resultado esperado:* la cuenta queda creada y el perfil refleja los cambios.

Capa 2 — detalle técnico.

1. `POST /users` llega al gateway con el cuerpo JSON.
2. El gateway delega en **`Users.CreateUser`** por gRPC (una sola llamada).
3. Users valida el formato (`username` de 3 a 50 caracteres, `email` con forma de email, `display_name` de 1 a 80) y normaliza los espacios.
4. Persiste con `INSERT INTO users ... RETURNING`; la base genera el `user_id`. Si viola `users_username_key` o `users_email_key`, el store traduce el error a `AlreadyExists`.
5. El usuario vuelve completo y el gateway responde **`201 Created`**.

Para la edición: `PATCH /users/{user_id}` → **`Users.UpdateUser`** con solo los campos enviados → `UPDATE` sobre `users` (que refresca `updated_at`) → `200`. El `username` no forma parte del mensaje de actualización: es inmutable.

Persistencia: `users_db · users`.

UC-2 · Seguir a otra persona y generar la alerta

Capa 1 — vista de usuario. Alicia sigue a Bernardo. Bernardo recibe una notificación de **nuevo seguidor** que, además del identificador, muestra **quién** la originó. *Resultado esperado:* el seguimiento queda registrado y Bernardo ve la alerta.

Capa 2 — detalle técnico. Este caso cruza **dos servicios y dos bases** y es el único flujo con una llamada de servicio a servicio.

1. `POST /users/{bob}/follow` llega al gateway con el encabezado `X-User-Id: alice`. El `{user_id}` de la ruta es **a quién** se sigue; **quién** sigue viene en el encabezado.
2. El gateway delega en **`Relationships.FollowUser(alice, bob)`** por gRPC.
3. Relationships valida que no sea un auto-seguimiento y hace `INSERT INTO follows ... ON CONFLICT DO NOTHING RETURNING`. **La arista nueva se detecta por el `RETURNING`:** si no devuelve fila, el seguimiento ya existía.
4. **Solo si la arista es nueva**, Relationships llama a **`Notifications.NotifyNewFollower(alice, bob)`** por gRPC (llamada 2→3). Notifications persiste en su propia base (`INSERT INTO notifications ... type='NEW_FOLLOWER'`).
5. Si Notifications estuviera caído, **el paso 4 se degrada**: el seguimiento ya está persistido y solo se registra una advertencia; la respuesta al usuario sigue siendo un éxito.
6. El gateway responde **`201 Created`** con la arista.

Persistencia: `relationships_db · follows` **y** `notifications_db · notifications`. Orden de comunicación: cliente → Gateway → Relationships → Notifications.

UC-3 · Consultar el grafo de seguidores (paginado)

Capa 1 — vista de usuario. Se consulta **quién sigue** a una persona y **a quién sigue** esa persona, por páginas. *Resultado esperado:* la lista de identificadores de la página pedida y el total, para poder mostrar "N seguidores".

Capa 2 — detalle técnico.

1. `GET /users/{id}/followers?limit=20&offset=0` llega al gateway.
2. El gateway delega en **`Relationships.GetFollowers(user_id, limit, offset)`**.
3. Relationships ejecuta dos consultas sobre su base: la página (`SELECT follower_id ... WHERE followee_id=$1 ORDER BY created_at DESC LIMIT $2 OFFSET $3`, cubierta por `idx_follows_followee`) y el total (`SELECT count(*) ...`). El `total` es independiente de la página.
4. Devuelve `user_ids` y `total`; el gateway responde **`200`**.

`GET /users/{id}/following` es simétrico, apoyado en la clave primaria compuesta. *Persistencia:* `relationships_db · follows`. **Nótese que Relationships solo conoce identificadores**: no guarda perfiles. Si un cliente necesita los nombres, sería el gateway quien los agregaría pidiéndolos a Users, exactamente con el mismo patrón que en UC-4 (una sola llamada de servicio por identificador distinto).

UC-4 · Ver la bandeja agregada y marcar como leída

Capa 1 — vista de usuario. Bernardo abre sus notificaciones y ve las alertas **sin leer**, cada una con **quién** la originó, con un único llamado. Luego marca una como leída y desaparece de la bandeja. *Resultado esperado:* una bandeja completa y autocontenida, y la alerta marcada.

Capa 2 — detalle técnico. Este es el caso donde el **gateway agrega**.

1. `GET /notifications` llega con el encabezado `X-User-Id: bob`.
2. El gateway delega en **`Notifications.GetUnreadNotifications(bob, limit, offset)`** (llamada 1).
3. Notifications consulta su índice `(user_id, is_read, created_at DESC)` y devuelve las alertas sin leer con su `total`.
4. El gateway reúne los `source_user_id` **distintos** y, **en paralelo**, llama a **`Users.GetUser`** una sola vez por cada uno (llamada 2..n) para **hidratar** el emisor.
5. Combina todo en una única respuesta `200` con `notifications` y `total`, donde cada alerta incluye su `source_user`.
6. Para marcar como leída: `POST /notifications/{id}/read` → **`MarkAsRead`** → `UPDATE notifications SET is_read=true WHERE id=$1 AND user_id=$2` → `200`.

Persistencia: `notifications_db · notifications`, enriquecida con lectura de `users_db` a través de su servicio dueño. **Ningún servicio lee la base del otro:** la agregación ocurre en el gateway, y un servicio caído solo degrada el campo accesorio (ver sección 7), nunca tumba la respuesta completa.

Otros flujos cubiertos

- **Dejar de seguir** (UC-2 inverso): `DELETE /users/{id}/follow` → `Relationships.UnfollowUser` → `DELETE FROM follows` → `204`; `404` si la arista no existía.
- **Consultar un perfil**: `GET /users/{id}` → `Users.GetUser` → `200`/`404`.

---

6. Decisiones técnicas y trade-offs

Cada decisión se presenta con su contexto, lo que **se gana**, lo que **se pierde** y las **alternativas descartadas**.

6.1 Límites de los servicios: por qué Users, Relationships y Notifications son tres servicios y no uno

Contexto. El dominio tiene tres responsabilidades con **ciclos de vida y patrones de acceso distintos**: la identidad (lecturas por clave, escrituras con unicidad estricta), el grafo de seguimiento (relaciones N:M de alta escritura y lecturas paginadas) y las alertas (escrituras disparadas por otros servicios y lecturas por bandeja).

Decisión. Un servicio por responsabilidad, cada uno con su propia base.

Ganancias. Cada uno tiene **cohesión alta** y un único motivo para cambiar; se escalan y despliegan por separado (un pico en el grafo no degrada la identidad); y el **fallo se aísla**: si Notifications cae, seguir a alguien sigue funcionando.

Pérdidas. Tres despliegues, tres bases que mantener y **sin transacciones cruzadas**: la consistencia entre el seguimiento y su alerta es *eventual*, no atómica.

Alternativas descartadas. Un **monolito modular** (más simple y con transacciones, pero no demuestra la arquitectura pedida y acopla todo el despliegue); **partir por conveniencia de trabajo** (rechazado explícitamente: la división debe responder a cohesión, no a repartirse tareas); y **un servicio de "actividad" continuo**, descartado porque diluye el límite de "notificaciones".

6.2 Una base de datos por servicio (y no una base compartida)

Decisión. Cada servicio tiene su **propia base**, con su **propia instancia y credenciales**; ninguno consulta la base de otro.

Ganancias. Es lo que hace **real** la independencia: cada servicio evoluciona su esquema sin coordinar; un fallo de una base no arrastra a las demás; y se elimina el *coupling* negativo que la rúbrica penaliza (servicios que comparten esquema quedan atados por él).

Pérdidas. No hay JOINs entre servicios ni transacciones distribuidas; hay que **agregar** en el gateway y aceptar duplicación de identificadores; y hay más infraestructura (tres instancias en vez de una).

Alternativas descartadas. Una sola base con tres esquemas (más cómoda y con transacciones, pero es exactamente el acoplamiento por base de datos que la rúbrica declara insuficiente); y **una base con una tabla por servicio** (peor aún). **Nota de implementación:** en el compose se eligió **una instancia por servicio** en lugar de un único motor con tres bases, para que la propiedad quede explícita en `docker ps` y para aislar el fallo; el costo es algo más de memoria, aceptable en el alcance del laboratorio.

6.3 PostgreSQL para Users, Relationships y Notifications

Decisión. Las tres usan **PostgreSQL**.

Ganancias. Las tres necesitan lo que una base **relacional** da gratis y bien: **restricciones** que son reglas de negocio en la base (unicidad de `username` y `email`, `CHECK` de no auto-seguimiento, `CHECK` de la forma de la notificación), **clave primaria compuesta** para el grafo (que además

impide seguir dos veces), **transacciones** en cada escritura, `TIMESTAMPTZ`, índices parciales y `gen_random_uuid()` para generar identificadores sin coordinación.

Pérdidas. Esquema rígido (todo cambio es una migración) y menos natural para documentos anidados.

Alternativas descartadas. MongoDB para estas tres: el grafo de seguimiento es *literalmente* una relación N:M con una restricción de unicidad y otra de no-reflexividad —justo el terreno donde un modelo de documentos obliga a implementar a mano lo que el motor ya garantiza—; la identidad exige unicidad estricta; y las notificaciones se consultan por bandeja ordenada, algo que un índice relacional resuelve directamente. Se reserva **MongoDB para Posts y Likes** (colecciones de documentos con contenido libre) y **Redis para el Feed** (sorted set ordenado por tiempo), donde cada tecnología aporta algo que las otras no.

6.4 gRPC con Protobuf como comunicación interna (y REST solo en el borde)

Decisión. REST/HTTP **solo** hacia afuera (en el gateway); gRPC/Protobuf entre todos los servicios.

Ganancias. El contrato es **tipado y versionado** en un `.proto` y genera stubs: un cambio incompatible **no compila**, en lugar de descubrirse en producción; los mensajes tienen **sentido de negocio**; y el transporte es eficiente sobre HTTP/2.

Pérdidas. Es **menos cómodo de depurar a mano** que un `curl` (mitigado: los servicios registran *server reflection*, así que `grpcurl` funciona sin los `.proto`) y no lo consume un navegador directamente (por eso existe el gateway).

Alternativas descartadas. REST entre servicios (más familiar, pero contratos más débiles y sin generación de código); **GraphQL** (innecesario aquí); **JSON-RPC** (sin tipado fuerte).

6.5 Sin broker ni cola de mensajes

Decisión. Todas las llamadas entre servicios son gRPC directo. No hay broker.

Ganancias. Cumple el requisito de la entrega (**una cola transportando tráfico entre servicios lo contradiría**), evita una pieza de infraestructura que mantener, y hace el flujo **tratable**: el orden de las llamadas se lee en el código.

Pérdidas. Las llamadas son **síncronas**: el emisor espera, y no hay almacenamiento intermedio que absorba picos. La consecuencia se asume explícitamente en UC-2: si Notifications está caído cuando alguien sigue a otro, la alerta **no se reintenta** (el seguimiento sí se persiste). Es una decisión consciente: se prefirió un sistema sin dependencias extra a una garantía de entrega que exigiría *outbox* o una cola.

Alternativas descartadas. RabbitMQ entre Posts y Notifications: figuraba en un borrador anterior y se **eliminó** porque contradecía el requisito de gRPC. **Patrón outbox + reintentos**: descartado por alcance (añade una tabla de eventos y un proceso de reenvío que no aporta a lo que evalúa esta entrega), anotado como mejora futura.

6.6 El API Gateway como único punto de entrada y agregador

Decisión. El gateway es la única superficie REST y, además, **agrega**: en `GET /notifications` combina Notifications con Users en una sola respuesta.

Ganancias. Único punto de entrada (los servicios internos quedan inaccesibles desde fuera, requisito de la rúbrica); los clientes hacen **una** llamada en vez de dos o tres y no necesitan conocer

la topología; los servicios quedan **desacoplados** entre sí (es la razón de que **no** exista una llamada Likes→Posts ni Notifications→Posts: el gateway resuelve esos campos).

Pérdidas. Un **salto extra** de latencia y un componente que puede volverse un **punto único de fallo** y un cuello de botella (mitigado: puede escalarse horizontalmente al ser sin estado; "no guarda nada"). También concentra complejidad: hay que cuidar que **no** se filtre lógica de negocio hacia él (por eso solo traduce y agrega).

Alternativas descartadas. Que los clientes consuman gRPC directamente (prohibido por el enunciado, y expondría los servicios internos); **un BFF por tipo de cliente** (sobre-ingeniería para esta entrega).

6.7 Diseño del Protobuf

Decisiones y por qué. El **tipo de notificación es un** `enum` y no un `string`: el contrato queda **cerrado** y el compilador rechaza valores inventados (antes era un `string` que permitía cualquier cosa). Los campos editables de `UpdateUser` son `optional`, que expresa con precisión la semántica de parche ("solo lo enviado"). Cada operación tiene **su propio mensaje de Request/Response** en lugar de reutilizar uno genérico: permite que cada uno evolucione sin arrastrar a los demás. Las fechas usan `google.protobuf.Timestamp`. Los identificadores son `string` en el contrato aunque la base use `UUID`.

Trade-offs. Un `enum` exige **actualizar el contrato (y regenerar) para añadir un tipo**, mientras que un `string` sería más laxo; se prefiere el error temprano. `optional` añade **presencia explícita** al JSON/binary, con algo más de tamaño. Muchos mensajes implican **más líneas** de `.proto`, a cambio de compatibilidad. `Timestamp` no es texto legible, pero es **inequívoco** y portable.

Alternativas descartadas. `string type` (acepta basura); un único mensaje "UserPatch"; fechas como `string` (ambiguas en zona horaria y formato).

6.8 Identificadores UUID generados por la base

Decisión. El `user_id` (y el `id` de notificación) son **UUID generados por la base** con `gen_random_uuid()`.

Ganancias. Los identificadores **no pueden colisionar** entre servicios ni dependen de coordinación; son **opacos** (no filtran cuántos usuarios hay) y el cliente **no** puede fabricar uno válido.

Pérdidas. Un UUID ocupa más que un entero y su aleatoriedad **penaliza la localidad** del índice (mitigado con UUID v4 en la práctica y volúmenes pequeños).

Alternativas descartadas. Autoincremental: enumerable (un cliente adivina identificadores ajenos) y colisiona entre servicios. **Generado por el cliente**: implica confiar en el cliente. **Siempre por el servicio en código**: posible, pero se prefirió el valor por defecto de la base para que ninguna ruta pueda insertar sin identificador.

6.9 Migraciones embebidas, incrementales y hacia adelante

Decisión. Cada servicio lleva sus migraciones **dentro de su artefacto** y las aplica al arrancar. Una migración ya aplicada **no se reescribe**: se corrige con una nueva.

Ganancias. El servicio es **autocontenido** (no necesita una herramienta ni un paso manual); el arranque es determinista; y es seguro ante reintentos. El caso real se dio en Users: al añadir `display_name` y la unicidad de `username` se creó la migración **002** en lugar de editar la 001 ya desplegada, rellenando las filas existentes **antes** de exigir `NOT NULL`.

Pérdidas. No hay **migraciones reversas** (un *rollback* exige escribir una nueva) y el arranque paga el costo de aplicar lo pendiente.

Alternativas descartadas. Herramienta externa (`golang-migrate`, Atlas): otro binario que versionar; **scripts de `initdb` en compose**: acoplarían el esquema al compose y romperían el principio de que cada servicio es dueño de su base.

6.10 Resiliencia: tiempo límite, errores ordenados, reconexión y degradación

Decisión. El gateway usa **conexiones perezosas** (arranca aunque un servicio esté caído), un **tiempo límite por llamada**, y **traduce cada error gRPC a un código HTTP con sentido**. Además, dos comportamientos de degradación deliberados.

Ganancias. La caída de un tercero **no propaga** ("sin colapsar ni crashear", lo que pide la rúbrica): `GET /notifications` sigue en `200` y solo omite el campo accesorio `source_user`; `FollowUser` persiste aunque Notifications esté caído; y todo **se recupera solo** cuando el servicio vuelve, sin reiniciar el gateway.

Pérdidas. Degradar implica **respuestas parciales** (una alerta sin su emisor) y la traducción de errores añade una capa que hay que mantener. Un **ajuste fino** fue necesario: con los parámetros por defecto de gRPC el canal queda intentando conectar ~20 s, así que un servicio caído agotaba el plazo y devolvía **504** en vez del **503** que corresponde; se acotó el intento de conexión (`grpc.WithConnectParams`) para que el canal pase rápido a fallo y la llamada responda **503** en ~0,01 s.

Alternativas descartadas. Reintentos automáticos: descartados porque no todas las operaciones son idempotentes y un reintento silencioso puede duplicar efectos. **Circuit breaker**: valioso a mayor escala, prematuro aquí (con el fallo acotado y la recuperación automática ya cubierta).

6.11 Identidad por encabezado `X-User-Id` (sin autenticación)

Contexto. El diseño indica que en varias operaciones el `follower_id` / `user_id` se toma "del encabezado", pero **no nombra** el encabezado. La rúbrica declara la autenticación **opcional**.

Decisión. El encabezado se llama **`X-User-Id`** y transporta el identificador del usuario que actúa.

Ganancias. Aísla la identidad en un solo lugar y deja la puerta abierta a sustituir el encabezado por un token sin tocar los servicios internos (el gateway lo resuelve una vez).

Pérdidas. No hay verificación: un cliente puede declarar cualquier identidad. Es aceptable y **explícito** en esta entrega; se documenta como el límite principal a cubrir con autenticación real.

Alternativas descartadas. JWT (fuera de alcance y añade emisor y claves); **dejar la identidad al servicio interno** (esparciría el problema por todos).

6.12 Cumplir el diseño al pie de la letra vs. apartarse de forma documentada

Al contrastar la implementación con el documento de diseño aparecieron puntos donde una lectura literal era **contradictoria o inconveniente**. Se resolvieron así, de forma **consciente y documentada**:

Punto del diseño	Implementación	Por qué
`id` como PK de `users`	`user_id`	El nombre es **deliberado**: el contrato y las referencias de otros servicios hablan de `user_id`, y así el campo no se confunde con el `id` de otra entidad.
`VARCHAR(n)` en `users`	`TEXT` + validación en el servicio	En PostgreSQL `TEXT` y `VARCHAR` rinden igual; la longitud es **regla de negocio** y se valida en la capa de servicio (3–50 y 1–80). Se evita duplicar el límite en dos sitios.
`users` sin `updated_at`	Con `updated_at`	`UpdateUser` **devuelve** `updated_at`; sin la columna el dato sería inventado. Se añadió y se documenta.
`notifications.post_id` como `ObjectId`	`TEXT`	Las notificaciones viven en **PostgreSQL**; el `post_id` es una **referencia lógica** a Posts (Mongo). Guardar el identificador como texto es lo coherente y no crea dependencia de tipo.
Respuestas REST con `status (string)`	Códigos HTTP + recurso	Un `200` / `201` / `204` / `409` **es** el estado; repetirlo en el cuerpo duplica y desincroniza. Se devuelve el **recurso**.
`Follow` con `status (string)`	Idem: resultado `OK` / `NotFound` + recurso	Misma razón: el estado se representa con el resultado gRPC y el recurso.
Listados del grafo no figuraban	Se añadieron `/followers` y `/following`	Son lecturas naturales del grafo; **sin ellas no habría forma de consultarlo**. Adición, no sustitución.

Estas diferencias **no cambian la arquitectura**; se registran para que la evaluación pueda contrastar diseño e implementación sin sorpresas.

---

7. Resiliencia y manejo de errores

El sistema se diseñó para que **la caída de una pieza no tumbe el resto**.

- **Conexiones perezosas.** El gateway abre la conexión gRPC en la primera llamada: arranca aunque un servicio aún no esté listo.
- **Tiempo límite por llamada y por petición.** Un servicio lento no arrastra al gateway ni a los demás.
- **Errores ordenados.** Cada error gRPC se traduce a un código HTTP con sentido, y todos los cuerpos de error comparten la forma `{"error", "message"}`:

HTTP	`error`	Cuándo
400	`invalid_argument`, `invalid_body`, `invalid_query`, `missing_user`	Datos inválidos o falta `X-User-Id`
404	`not_found`	El recurso o la ruta no existe
405	—	Método no permitido en la ruta
409	`already_exists`	`username` o `email` duplicados
502	`bad_gateway`	El servicio devolvió un error inesperado
503	`service_unavailable`	El servicio de destino está caído
504	`deadline_exceeded`	El servicio no respondió a tiempo

  Los errores **internos no se filtran**: se registran con detalle en el log y al cliente se le da la forma genérica. Los de **validación** sí se propagan con su mensaje, porque son útiles y no revelan infraestructura.

- **Recuperación de panics** en el gateway: una petición no puede tumbar el proceso.
- **Degradación elegante** (dos comportamientos deliberados): la bandeja se entrega sin el campo accesorio si Users no responde; el seguimiento se persiste aunque Notifications esté caído.
- **Apagado ordenado**: los servicios drenan las peticiones en curso ante `SIGTERM`.

Evidencia de verificación. Sobre el sistema levantado con `docker compose`:

- **44 comprobaciones** end-to-end por HTTP contra el único puerto publicado, cubriendo las once rutas del gateway, con persistencia real y el disparador de notificaciones.
- **16 comprobaciones** de resiliencia **con contenedores reales**: con `users-svc` detenido, `GET /users/{id}` responde **503** y `GET /notifications` sigue en **200** (sin el emisor); el gateway responde `200` en `/health`; al volver el servicio todo se recupera **sin reiniciar el gateway**; y se comprobó la **persistencia** de los datos tras un `docker compose down` + `up` (volúmenes).
- **Ejemplos del README** ejecutados literalmente desde un sistema vacío (`down -v`), incluidas las migraciones.

- **Pruebas unitarias** de cada servicio (Go: `go test ./...`; Notifications: `pytest`), y **pruebas de integración de la capa de datos contra un PostgreSQL real** para los tres servicios (`*_TEST_DB_DSN`), que ejercitan las migraciones embebidas y las consultas sobre la base, no solo dobles en memoria.

---

8. Cumplimiento de los requisitos de la entrega

Requisito	Estado
Repositorio Git público	✅ GitHub, `Makx3/Kite`, modelo Git Flow (`main` / `develop` / `feat/*`)
API Gateway como única superficie REST y único punto de entrada	✅ 11 rutas; solo el 8080 publicado
Servicios internos inaccesibles desde fuera de Docker	✅ Red `backend` interna; 50051–50053 cerrados al host (verificado)
Servicio de Usuarios	✅ `users-svc`
≥ 3 servicios de negocio (además del Gateway y de Usuarios)	⏳ 2 en funcionamiento (Relationships y Notifications); **Posts**, **Feed** y **Likes** están descritos en el diseño y son los que completan el mínimo
≥ 2 servicios de negocio en Go	⏳ 1 en funcionamiento (Relationships); **Posts** y **Feed** (ambos en Go) están en el diseño y completan el mínimo
Comunicación interna por gRPC/Protobuf	✅ Todos los contratos `.proto` versionados
Cada servicio en su propia imagen Docker	✅ Cuatro imágenes
`docker compose up` levanta el sistema	✅ Verificado desde cero
Comunicación por nombre de servicio, sin IPs fijas	✅ Sin IPs en ninguna parte
`.env` con valores de ejemplo	✅ Commiteado y leído por Compose
Persistencia real (nada importante en memoria)	✅ PostgreSQL con volúmenes; supervivencia verificada
Justificación tecnológica de cada BD	✅ Sección 6.3
≥ 4 casos de uso end-to-end	✅ UC-1..UC-4 (+ flujos adicionales)
≥ 2 entidades de dominio persistidas	✅ `User`, `Follow`, `Notification`
Configuración por variables de entorno	✅ Validación al arrancar, sin valores silenciosos
Separación en capas por servicio	✅ Transporte / negocio / acceso a datos

9. Cómo levantar y probar el sistema

La guía operativa completa —puesta en marcha, tabla de variables, `curl` de cada endpoint, un flujo completo de una pasada y la prueba de resiliencia paso a paso— está en el **`README.md`** del repositorio. En resumen:

```
docker compose up -d --build          # levanta el sistema completo
curl -s localhost:8080/health         # comprueba que responde
```

---

10. Estado y trabajo pendiente

Implementado y verificado de extremo a extremo en esta entrega: Users, Relationships, Notifications y el API Gateway, con `docker compose`, `.env`, volúmenes y el README operacional.

Pendiente (siguiente iteración, sobre la misma arquitectura): los servicios **Posts** (MongoDB), **Feed** (Redis) y **Likes** (MongoDB), y con ellos sus rutas en el gateway (`POST /posts`, `GET /posts/{id}`, `POST|DELETE /posts/{id}/like`, `GET /feed`), que ya están descritas en el documento de diseño. El contrato de Notifications ya contempla el disparador `NotifyPostLiked`, de modo que el servicio de Likes podrá usarlo sin tocar notificaciones. El **Feed** es el que introducirá una tercera tecnología (Redis) para el muro por usuario, y el **Gateway** seguirá siendo el punto donde se agregue el contenido de Posts con los identificadores del Feed.

Mejoras anotadas: autenticación real que sustituya el encabezado `X-User-Id`; *outbox* con reintentos si se quisiera garantizar la entrega de notificaciones ante caídas; y migraciones reversas.